Homework 3

Danen M. Pruitt

ENS, AFIT

OPER 646: Reinforcement Learning

May 2026

Contents

1 Introduction

In this write up we will be looking at the differences between application of on-policy and off-policy Monte Carlo Control (MCC) to the Farama Foundation Gymnasium *CartPole-v1* environment which consists of a pole hinged on a cart that can be moved left or right. The goal is to keep the pole vertical at each step and the agent is rewarded for every step the pole remains reasonably upright. The application of on-policy and off-policy MCC to this environment helps illustrate the differences between these approaches particularly with regards to policy imporvement and convergence.

1.1 Implementation

The implementations in this write up utilize Python and the OpenAI Gymnasium library to construct the environment and deploy the Monte Carlo Control (MCC) algorithms. While the full code is available in the accompanying files, several implementation details will be highlighted here. Both the on-policy and off-policy MCC experiments were derived from Handout 8, and were subsequently modified into callable functions within the $Homework\_3.py$ script. The experimental setup consisted of 60 design runs, comprised of 10 replications with 500 episodes per replication. To monitor performance, every 25th training episode was designated as a milestone and evaluated using the following code block:

```
if m % test_freq == 0:
            mean, hw = evaluate_policy_test(Q, phi, env, num_test_reps)
            GzmTest.append((z, m, mean, hw))
            update_best_scores(mean, hw, Q, best_scores)
```

The *evaluate_policy_test* and *update_best_scores* functions are previously defined helper functions to aid in the readability and efficiency of the script. These evaluated values are then used to compute our $MeanMaxEETDR$ and $MeanTime - AvgEETDR$ as well as form the 95% confidence intervals. Both implementations used seeds for reproducibility but rather than setting static seeds for entire scripts, they were defined by function and use. In the $LHC\_tuning\_on/off.py$ scripts, the variable *seed* was set equal to 1 to control the Latin Hypercube design sampling within the $LatinHypercubeSampling()$ function. Within the $mcc\_experiment\_on/off\_policy()$ functions, theseeds were set for both numpy and the environment as $np.random.seed(int(z * 1e6 + m + offset))$ and $env.reset(seed = int(z * 1e6 + m + offset))$ where z is the replication index, m is the episode index, and $offset$ which was se to zero for these experiements. The combined formula ensures unique seeds per episode while still being reproducible across multiple runs. The same is true in the seed for our $evaluate\_policy\_test()$ function with $seed = seed\_mult * 1000 + rep$ where *seed_mult* is 1 by default and 2 when testing the superlative policy and rep is the test episode index.

2 On-Policy MCC

In on-policy MCC, the agent learns about a policy while following that same policy. The agent generates episodes by interacting with the environment using the current policy, and then updates the action-value function based on the returns received from the episodes experienced. The on-policy MCC algorithm can be summarized as follows:

1. **Algorithm parameter:** small $\epsilon > 0$
2. **Initialize:**
   - $\pi \leftarrow$ an arbitrary ϵ-soft policy
   - $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
   - $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
3. **Repeat forever (for each episode):**
   (a) Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
   (b) $G \leftarrow 0$
   (c) **Loop for each step of episode,** $t = T-1, T-2, \ldots, 0$**:**
      i. $G \leftarrow \gamma G + R_{t+1}$
      ii. **Unless** the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
         - Append G to $Returns(S_t, A_t)$
         - $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
         - $A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)
         - For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \epsilon + \epsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \epsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

2.1 Hyperparameter Tuning Results

The primary hyperparameters in question for us are $\bar{Q}_0$ the optimistic initial action-value estimate, ϵ_a the initial exploration rate, and ϵ_b the terminal exploration rate. Below is a table displaying the OLS and ANOVA results for the on-policy implementation.

From these results we can conclude that *eps_a* has the most impact on algorithm perfomance. This is very evident from the F statistic of 7.5417 in the ANOVA results and the high coefficient of 259.267. That high coefficient tells us that for each unit increase in *eps_a*, the average reward per episode increases

Table 1: Combined Regression and ANOVA Results

Part 1: OLS Regression Results						
Variable	Coef.	Std. Err.	t	$P > \|t\|$	[0.025	0.975]
eps_a	259.2670	94.409	2.746	0.008	69.641	448.893
eps_b	-200.2917	83.928	-2.386	0.021	-368.866	-31.718
Init_Qbar	146.2779	94.504	1.548	0.128	-43.538	336.094

Part 2: ANOVA Results (Type II Sum of Squares)				
Variable	Sum Sq.	df	F	$PR(>F)$
eps_a	16830.248066	1.0	7.541707	0.008357
eps_b	12709.718943	1.0	5.695280	0.020836
Init_Qbar	5346.642870	1.0	2.395854	0.127964

by that amount making it a very strong predictor of performance.

According to the ANOVA results, *eps_b* also has a significant impact on the performance of the model, but the OLS results show that it is a negative impact. This makes sense because as we develop the algorithm, we want to reduce the amount of exploration as we get closer to the terminal state. Thus if our exploration rate continues to be high, the model will continue to explore and perform suboptimally. According to both ANOVA and OLS results, $\bar{Q}_0$ is not extremely statistically significant with P-values of 0.128 in both cases. However, there is apositive contribution to performance denoted by the positive coefficient of 146.2779. Though it is not as strong of a predictor it does still contribute to performance and should be condisdered when tunining the algorithm.

2.2 Design Run Results

Below is a table of the top 10 algorithm desighm runs.

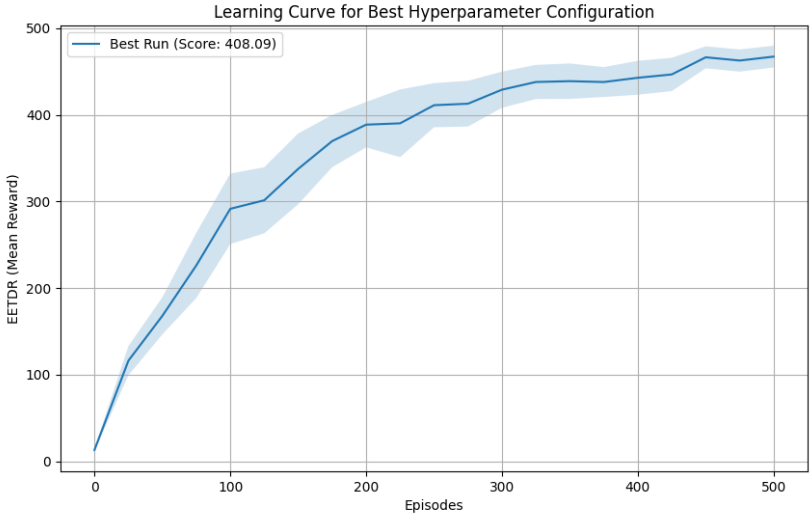
Table 2: Top 10 Algorithm Design Runs by Score (LHD Experiment)

Run	**Alg.**	**Time**	**Max EETDR**		**Time-Avg EETDR**		**Superlative Policy**	
Idx	**Score**	**(sec)**	**Mean**	**95ME**	**Mean**	**95ME**	**EETDR**	**95% HW**
42	408.09	254.587	479.49	14.84	360.74	37.49	500.00	0.00
22	398.50	261.863	479.58	21.54	354.00	44.81	500.00	0.00
7	388.10	230.688	469.08	31.18	363.94	50.53	500.00	0.00
55	382.05	230.679	471.11	37.03	348.74	44.73	500.00	0.00
12	382.05	248.933	463.73	40.09	367.11	47.45	500.00	0.00
59	368.37	227.659	459.91	30.94	309.44	31.97	500.00	0.00
57	366.27	190.129	466.01	34.08	305.44	37.67	500.00	0.00
21	348.72	166.302	439.18	51.59	336.89	46.49	500.00	0.00
18	340.02	230.659	441.65	47.01	296.74	38.65	499.47	1.07
39	335.66	305.079	426.16	65.18	353.04	55.36	500.00	0.00

Note: Time (sec) represents mean execution time per replication. 95ME = 95% Margin of Error; HW = Half-Width. Total experiment time: 2063.955s.

Here, the highest algorithm score was 408.09 with an execution time of 254.587 seconds. The superla-

tive policy had an EETDR of 500.00 with a half-width of 0.00 which is not uncommon in our experiment runs likely because at least one policy was able to "get lucky" at least once and max out the reward. Below is the learning curve for the best design run.



3 Off-Policy MCC

In off-policy MCC, the agent learns about a target policy while following a different behavior policy. The agent generates episodes by interacting with the environment using the behavior policy, and then updates the action-value function for the target policy using importance sampling to weight the returns received from the episodes experienced. The off-policy MCC algorithm can be summarized as follows:

1. **Initialize, for all** $s \in \mathcal{S}, a \in \mathcal{A}(s)$**:**
   - $Q(s,a) \in \mathbb{R}$ (arbitrarily)
   - $C(s,a) \leftarrow 0$
   - $\pi(s) \leftarrow \arg\max_a Q(s,a)$ (with ties broken consistently)
2. **Repeat forever (for each episode):**
   (a) $b \leftarrow$ any soft policy
   (b) Generate an episode following b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
   (c) $G \leftarrow 0$
   (d) $W \leftarrow 1$
   (e) **Loop for each step of episode,** $t = T-1, T-2, \ldots, 0$**:**
      i. $G \leftarrow \gamma G + R_{t+1}$

ii. $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$

iii. $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$

iv. $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken consistently)

v. **If** $A_t \neq \pi(S_t)$ **then** exit inner Loop (proceed to next episode)

vi. $W \leftarrow W \frac{1}{b(A_t|S_t)}$

One of the primary differences between the on-policy and off-policy implementations is the use of weighted importance sampling to converge to the optimal policy. This also means that the method only learns at the tails of episodes which could lead to slower learning.

3.1 Hyperparameter Tuning Results

Again, the primary hyperparameters in question for us are $\bar{Q}_0$, ϵ_a, and ϵ_b. Below is a table displaying the OLS and ANOVA results for the off-policy implementation. Different from our on-policy results,

Table 3: Combined Regression and ANOVA Results

Part 1: OLS Regression Results						
Variable	Coef.	Std. Err.	t	$P > \|t\|$	[0.025	0.975]
eps_a	-49.563	20.218	-2.451	0.018	-90.172	-8.953
eps_b	109.828	17.974	6.110	0.000	73.726	145.929
Init_Qbar	37.211	20.239	1.839	0.072	-3.440	77.861

Part 2: ANOVA Results (Type II Sum of Squares)				
Variable	Sum Sq.	df	F	$PR(> F)$
eps_b	3821.489	1.0	37.337	¡.001
eps_a	615.038	1.0	6.009	0.018
Init_Qbar	345.990	1.0	3.380	0.072

eps_b is the most significant hyperparameter by a significant margin. Since off-policy learns from the results of a behavior policy, that policy must have coverage. A higher *eps_b* ensures that every possible state-action pair the target policy could visit is visited by the behavior policy and therefore the stong influnce demonstrated by our above results is to be expected.

3.2 Design Run Results

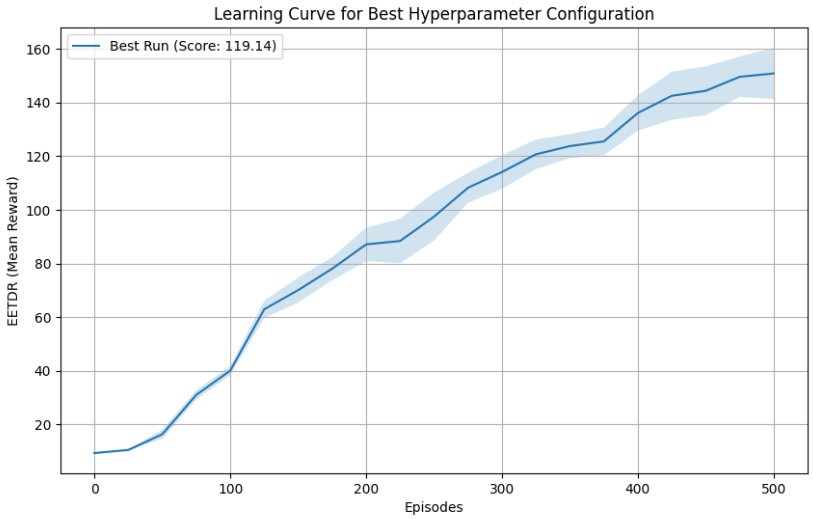
Below is a table of the top 10 algorithm desighm runs.

Table 4: Top 10 Algorithm Design Runs by Score (LHD Experiment)

Run	Alg.	Time	Max EETDR		Time-Avg EETDR		Superlative Policy	
Idx	Score	(sec)	Mean	95ME	Mean	95ME	EETDR	95% HW
48	119.14	64.237	157.39	16.25	91.38	5.25	202.15	6.35
33	111.50	61.563	171.11	43.04	96.75	10.13	315.92	31.02
42	111.14	65.966	154.82	20.26	79.12	3.11	187.57	38.81
2	110.92	16.149	137.08	11.13	94.63	6.28	160.30	6.28
28	108.16	65.979	148.36	24.45	89.63	5.07	259.35	21.37
41	107.50	62.930	152.29	28.44	90.61	7.63	250.03	11.83
39	106.55	62.803	139.70	12.92	81.05	4.86	173.40	11.67
37	106.54	67.640	142.28	22.66	92.05	5.12	247.72	12.37
47	106.26	66.696	130.72	10.58	90.05	4.59	167.23	5.28
58	105.48	27.473	135.70	16.46	90.00	5.15	261.53	25.43

Note: Time (sec) represents mean execution time per replication. 95ME = 95% Margin of Error; HW = Half-Width. Total experiment time: 539.827s.

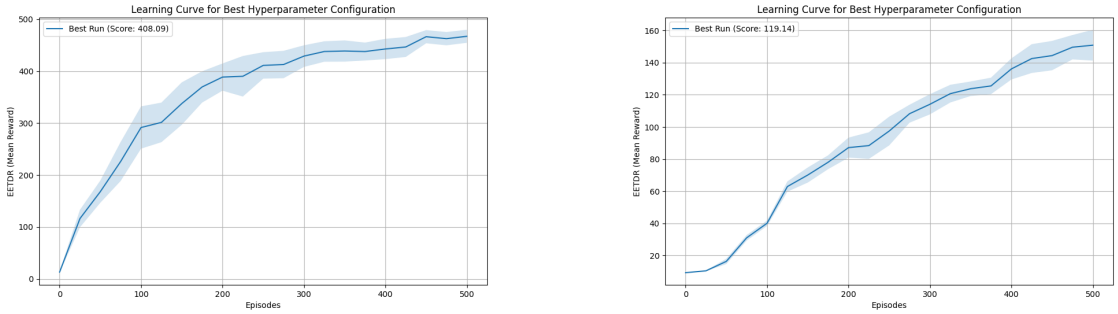
One major difference we see between on and off policy is the clock time each implementation takes. Off-policy MCC terminates early when a non greedy action is encountered resulting in the algorithim computing less per episode than on-policy MCC. We also see that this has an effect on our overall algorithm scores and the EETDR of the superlative policy. Below is the learning curve for the best design run found with off-policy MCC.



The EETDR reached here is much lower than the one found with on-policy MCC only reaching approximately 150. This gives credence to the claim that off-policy learns slower than on-policy. Given more runs results may have been different.

4 Algorithm Comparisons

Below are both learning curves.



(a) On-Policy Learning Curve

(b) Off-Policy Learning Curve

Figure 1: Side-by-side comparison of learning curves.

Though they have different scales on the vertical axis, it is clear that the on-policy learning curve plateaus at about $200-250$ episodes while the off-policy learning curve seems to still be rising at 500 episodes. We also notice that the on-policy confidence interval narrows while the off-policy confidence interval broadens. This is due to the source of stocasticity in the two implementations. On-policy's stocasticity comes primarily from the environment while off-policy's stocasticity comes from the application of weighted importance sampling. This increased varaince is another reason off-policy is a slower learner compared to on-policy.

Table 5: Top Algorithm Design Runs

Policy	**Alg.**	**Time**	**Max EETDR**		**Time-Avg EETDR**		**Superlative Policy**	
on/off	**Score**	**(sec)**	**Mean**	**95ME**	**Mean**	**95ME**	**EETDR**	**95% HW**
On	408.09	254.587	479.49	14.84	360.74	37.49	500.00	0.00
Off	119.14	64.237	157.39	16.25	91.38	5.25	202.15	6.35

Comparing the metrics between the two best design runs, we see that the superlative on-policy ,as noted before, was able to max out the reward as well as get close to maxing out in the $MaxEETDR$ column. The off-policy run however didn't even get a majority of the possible reward in any column.

For this specific problem the on-policy algorithm is the most appropriate and stronger option. The $CartPole-v1$ envoronment is a reward dense environment where the agent needs to learn a very delicate and specific policy to succeed. This does not lend itself well to the exploratory nature of the off-policy algorithm so the more strict on-policy algorithm is more appropriate and finds the optimal policy much faster than the off-policy algorithm.

5 Appendix

5.1 AI Usage

The use of generative AI was within class guidlines as outlined in the sylabus. Most frequently it was used to check sytax and coding logic during development by pasting given errors into the chat. In those cases where code was directly passed to the chat the phrase, " without editing the code..." was used. The heaviest use was in the creation of proper latex sytax for tables and fugures to ensure a clean and readable document. The primary model used was Gemini 3 with some use of Copilot.